ECE 532 Project Report

Overall Design

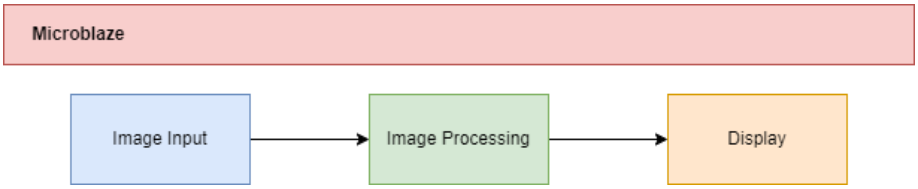

Figure 1. Overall Design Architecture

The project was split into 3 digital blocks, with the Microblaze being treated as a system management system. There was an image input block, an image processing block, and a display driver. Additionally, there were mechanical and electrical design elements in the project.

This split was convenient for efficient work allocation between group members, and made it easy to independently design and test blocks. It also enabled a scaling of complexity of each block as it became necessary with time constraints in the project.

My work was on the image processing block as well as the mechanical design of the project. As such, these are the sections that will include design process details below.

Mechanical and Electrical Design

Despite this being a digital design course, the project was heavy on mechanical design elements. Myself and another teammate were responsible for designing the slip-ring, which consisted of a set of motor brushes and copper tubes that were in turn soldered to the LED strips. The motor brushes would be mounted by the motor arm and be stationary, while the copper tubes which were rigidly attached to the fan arms would be spinning. The brush contact to the copper tube would ensure consistent conductivity and clean data transmission. The final iteration of this design can be seen below.

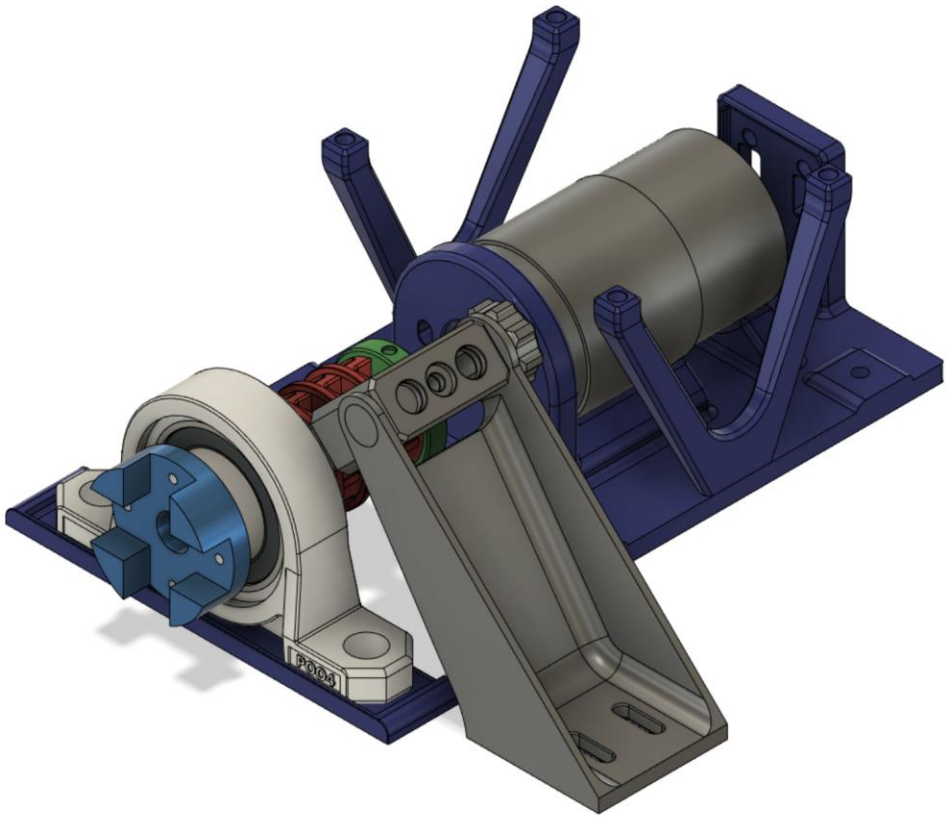

Figure 2. Final Iteration of the baseplate, metal core, and motor brush holders that were used in the final design of our project

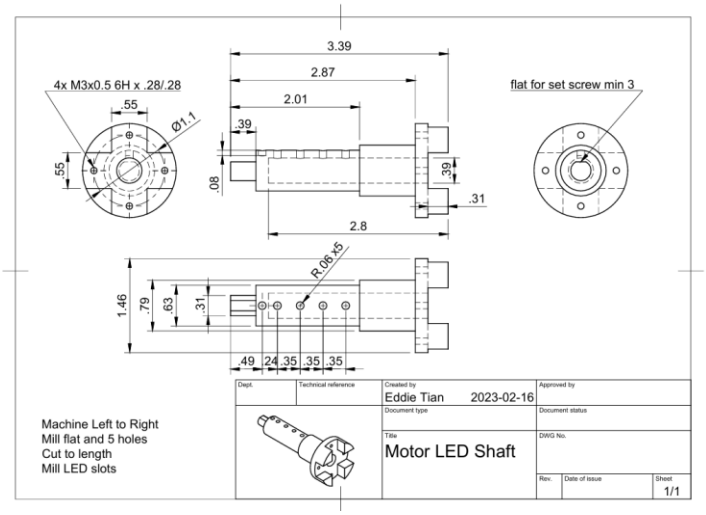

Figure 3. Metal Core Design

The front part which would have the fan arms screwed into it (also shown in Figure 8) was a metal core machined by a teammate. The rest of the design consisted of 3d printed parts designed by myself and another teammate. This mechanical and electrical design element was completed long before the hardware was, and the spinning fan arm was initially tested with an Arduino. The results of that test can be seen below.

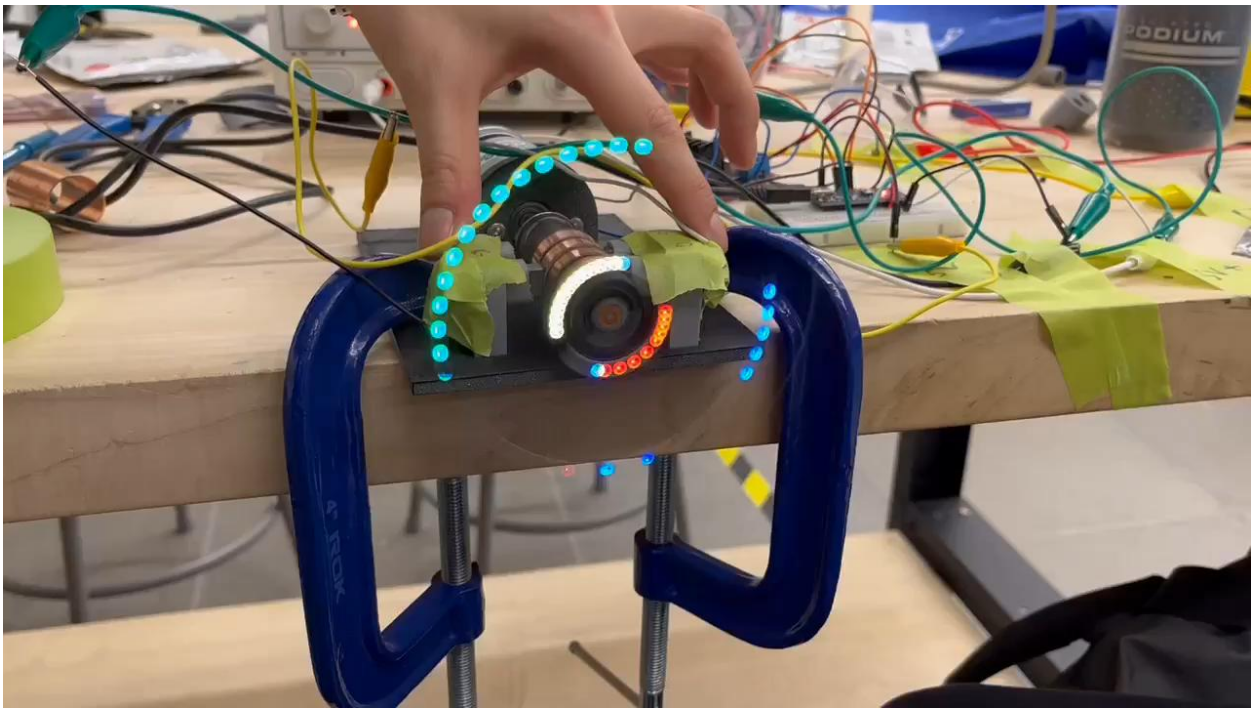

Figure 4. Testing the mechanical and electrical design with an Arduino

I was also responsible for designing and putting together the cabinet that would be used to hold this project, as we were concerned about this rather large and fast-spinning project hurting someone. Unfortunately, my initial design was too expensive to build due to MyFab wood panels being significantly more expensive than they were a couple of years ago, so we ended up purchasing an IKEA cabinet and making modifications to the shelf piece and adding a PETG front panel. These designs can be seen below.

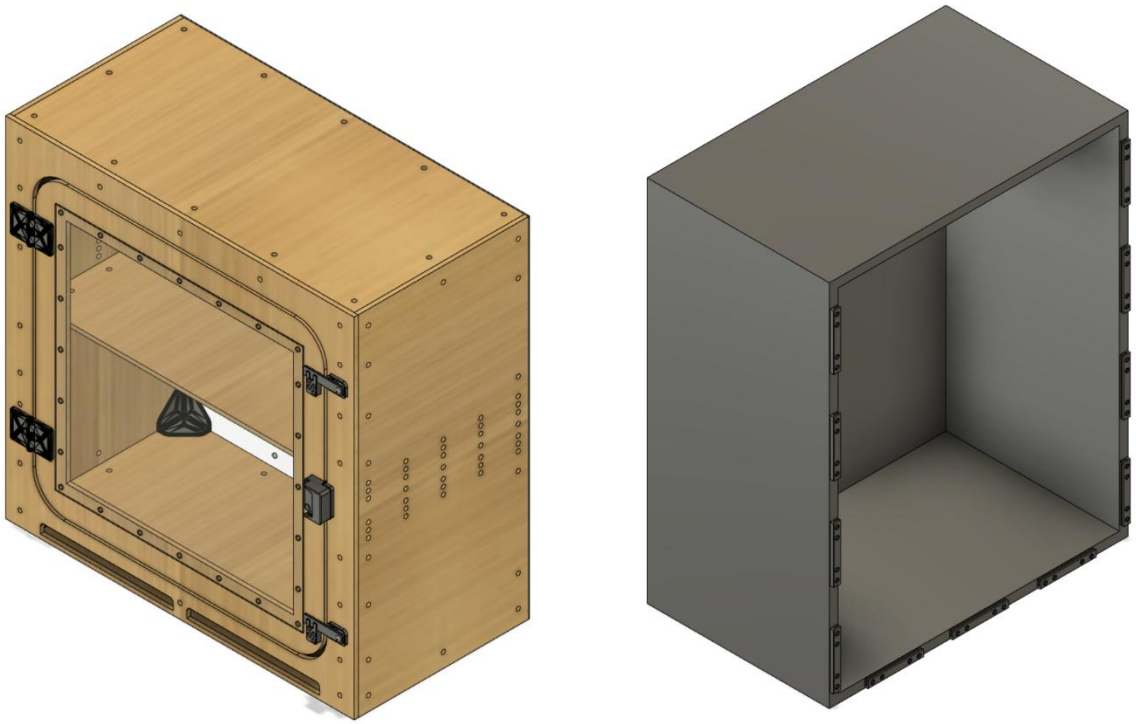

Figure 5. Enclosure design for the spinning LED display. Left is the original design that was unused, and right is the ikea cabinet with the slots designed to hold a PETG panel.

Image Input Block

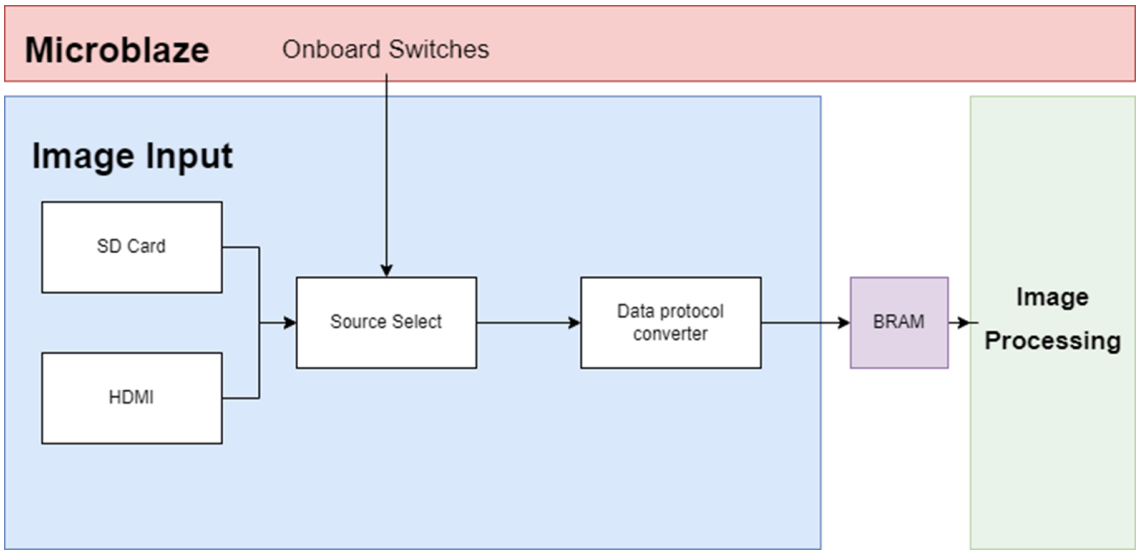

Figure 6. Image Input Block Implementation

Image Processing Block (mine)

The block I was responsible for designing was the image mapping block. The goal of this block was to take the input image and an input map, and output a new array which would have the image mapped to polar coordinates. This was necessary in order to send the pixel values to the LEDs on the arms in the correct order. An overview of the idea and the block can be seen below.

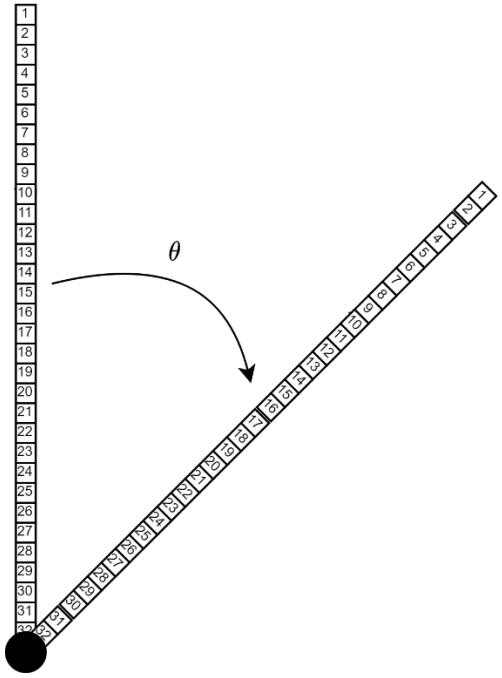

Figure 7. LED arm representation. 32 LEDs per arm, theta represents angle between LED arms when the arm is rotated.

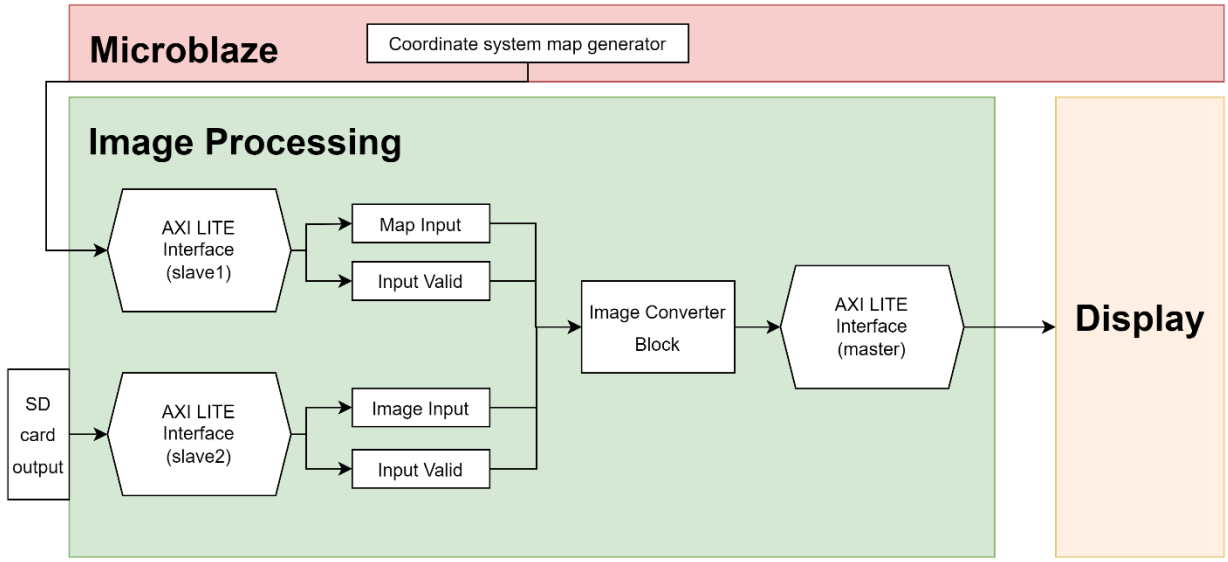

Figure 8. Overview of the Image Mapping Block

In order to design this block, I first determined the image map and how the conversion would look like using python. One variable that would impact how the output image would work would be how often we wanted to update the pixel values on each rotation. That is, I needed to decide that θ value would be between two LED arm positions, and thus how many pixel value updates I would send per rotation. Under the assumption of 180 updates per rotations ($\theta = 2°$), the following was generated in Python to verify the block would work.

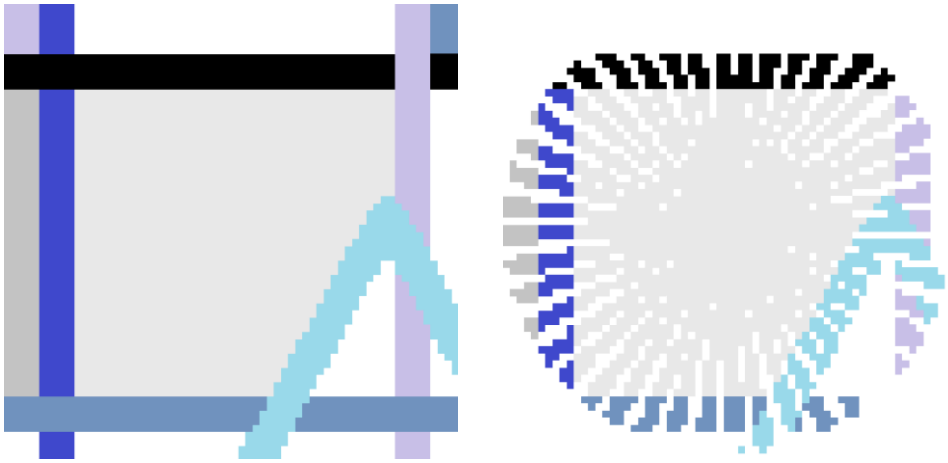

Figure 9. Image Converter Block test via Python. Left depicts the original image, right indicates recreated image with sampling every 2 degrees.

With this done, I could start implementing. I chose to generate the image map in C as this was a computation-heavy step that could be done once on the microblaze. In hardware, I would use this image map to pick RGB values that would be sent to the LEDs in the rotating arm based on the input image.

The next step was to determine how communications would work, and we opted to use AXI masters and slaves. My block needed input from both the Microblaze and SD card blocks, so I implemented 2 slaves. Since I was also sending large amounts of data and I wanted the map and input image to each have their own large register, I also implemented a bitshift in the slaves, where each slave would write the 32 bits received to the last 32 bits of each register, and then bitshift by 32 before the next 32 bits were written.

Finally, I also needed to implement one master to send data to the LED driver block. The master would wait until output valid was asserted, and then send the data 32 bits at a time.

All of the components above were designed individually and tested via simulation testbenches and looking at output waves in Vivado.

Display Driver Block

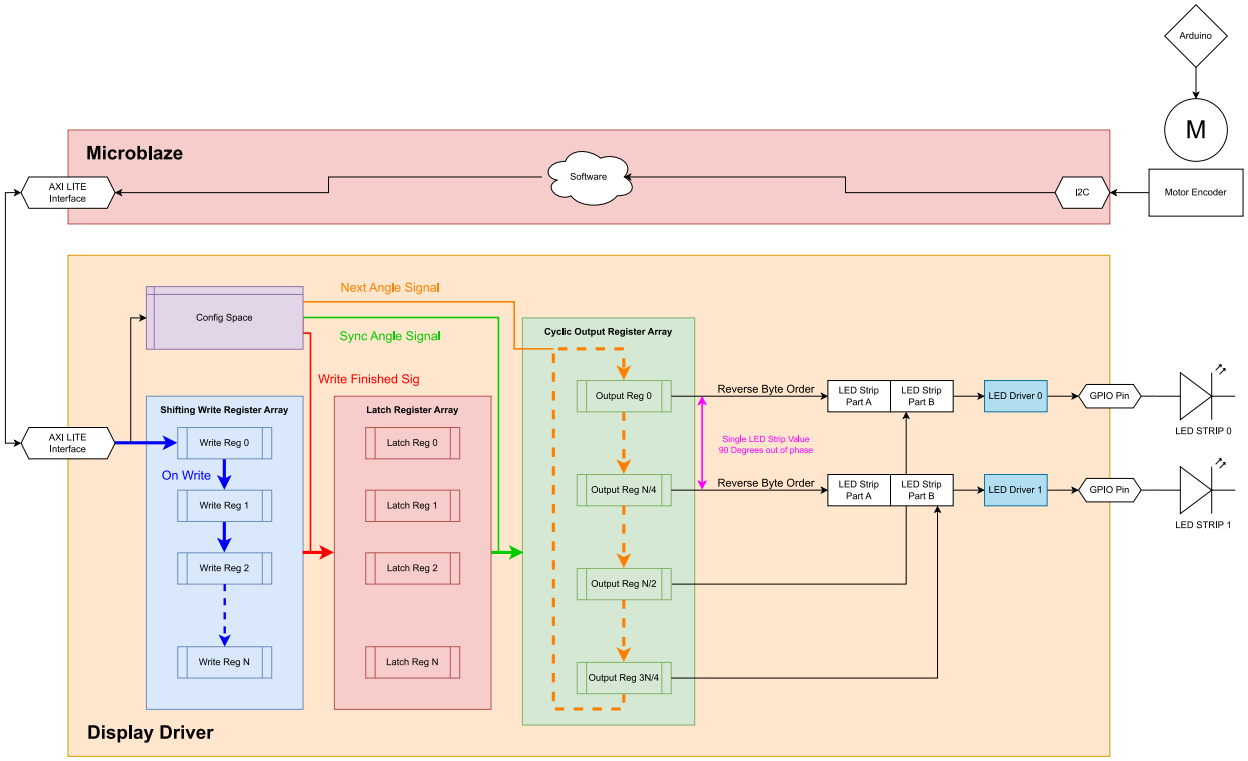

Figure 10. Display Driver Block Implementation